Unit-1 Introduction

Introduction:-

Software: - Software is set of tested and integrated programs plus documentation or collections of programs is called software.

Ex: Windows or any computer Applications.

or

The term software specifies to the set of computer programs, procedures and associated documents (Flowcharts, manuals, etc.) that describe the program and how they are to be used.

➢ Software is more than programs. Any program is a subset of software, and it becomes software only if documentation & operating procedures manuals are prepared.

There are three components of the software:

➢ **Program**: Program is a combination of source code & object code.

➢ **Documentation**: Documentation consists of different types of manuals.

Examples of documentation manuals are: Data Flow Diagram, Flow Charts, ER diagrams, etc.

➢ **Operating Procedures**: Operating Procedures consist of instructions to set up and use the software system and instructions on how react to the system failure.

Engineering:-

Engineering is a applications of science, tools & methods and also designing, testing & building of machines, structures & Process using math's & science.

Ex: Software Engineers, Civil Engineers, Mechanical Engineers & etc.

Software Engineering:-

Software Engineering is a detailed or systematic study of engineering to the design, development & maintenance of software is called software engineering.

Role of Software engineering:-

1) **Software as a Product**.
   - Software is a product, which is made to be end users.

     Ex: Computer Applications.

2) **Software as a vehicle for delivering a Product.**
   - It controls other programs.
   - Supports or directly provides system functionality.
   - Help to build other software.
   - Ex: Operating System, Software tools,

Professional software development:

Professional software development in software engineering refers to the disciplined and systematic process of designing, building, testing, and maintaining software systems by following industry for best practices, standards, and ethical guidelines.

- It includes techniques that support program specification, design, and evolution, none of which are normally relevant for personal software development.
- A professionally developed software system is often more than a single program.
- The system usually consists of a number of separate programs and configuration files that are used to set up these programs.
- It may include system documentation, which describes the structure of the system; user documentation, which explains how to use the system, and websites for users to download recent product information

It involves a combination of technical skills, collaboration and methodologies to deliver highquality software to the customer. The process of a software development has three generic views which are

1. **Definition Phase:-**

It is the base of definition phase; the experts get the knowledge about….

- Information needed for processing.
- Which functions are required?

2. **Development Phase:-**

In development phase different types of queries raised in developer mind…

- How to design the data structure & architecture of software.
- How design convert in a programming language.
- How to implementation code.
- Testing of software how to perform.

3. **Maintenance Phase:-**

The main focus of maintenance phase is changes which are…

- Correction of errors.
- Adaption of new idea.
- According to the needs of software after change in customer mood.

Need for Software Engineering

The need of software engineering arises because of higher rate of change in user requirement and environment on which the software is working.

The following factors contribute to the need of software engineering.

*** Large Software**: As the size of software becomes large, engineering has to step to give it a scientific process.

*** Scalability**: If the software process were not based on scientific and engineering concepts, it would be easier to re-create new software than to scale an existing one.

Cost: As hardware industry has shown its skills and huge manufacturing has lower down the price of computer and electronic hardware. But the cost of software remains high if proper process is not adapted.

*** Dynamic Nature**: The always growing and adapting nature of software hugely depends upon the environment in which user works. If the nature of software is always changing, new enhancements need to be done in the existing one. This is where software engineering plays a good role.

* **Quality Management**: Better process of software development provides better and quality software Product.

Characteristics of Software: Software products may be developed for a particular customer or may be developed for a general market.

➢ Software is engineered not manufactured.

➢ **Software is intangible** – It has no mass, no volume, no color, no odor and it cannot be touched. ➢ Software does not wear out, failed components must be re-engineered.

➢ **Maintainability** – Software should meet the changing needs of customers.

➢ **Usability** – The software should have appropriate user interface and enough number of documentations.

➢ **Dependability** – Means reliability, it should be safe and secure to use the software without causing system failure

Software Crisis: There are several various general models or paradigms of software development:

➢ **Size**: Software is becoming more expensive and more complex with the growing complexity and expectation out of software. For example, the code in the consumer product is doubling every couple of years.

➢ **Quality**: Many software products have poor quality, i.e., the software products defects after putting into use due to ineffective testing technique.

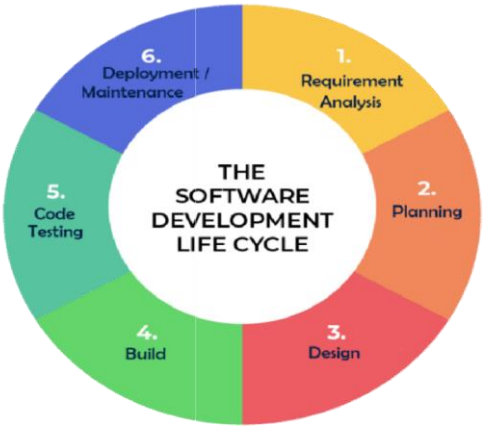
➢ **Cost:** Software development is costly i.e. in terms of time taken to develop and the money involved.

➢ **Delayed Delivery**: Serious schedule overruns are common. Very often the software takes longer than the estimated time to develop, which in turn leads to cost shooting up.

Software development life cycle model

Software development life cycle model is a key concept of Software engineering.

It is generally consists of series of stages



- Requirement Analysis
- Planning
- Design
- Coding & Implementation
- Code Testing
- Deployment & Maintenance

1. **Requirement Analysis**:
   - Requirements analysis is the process of determining user expectations for a new or modified product.
   - The requirements must be quantifiable, relevant and detailed.

2. **Planning:**
   - The purpose of the second stage is to outline the scope of the problem and identify solutions. Resources, costs, time, and other aspects should be considered here.
   - The planning phase of the SDLC is also when the project plan is developed that identifies and assigns the tasks and resources required to build the structure for a project.

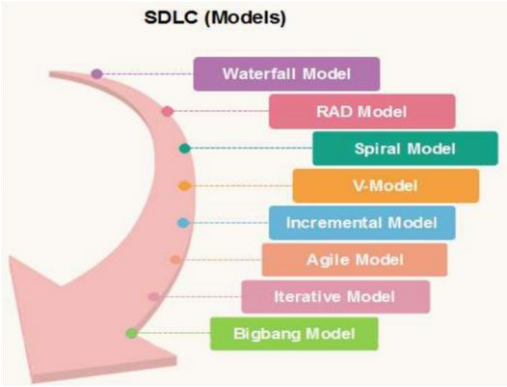
3. **Design:**
   - The system design helps in specifying overall system architecture, ER-diagram, DFD, UML Diagram etc.

4. **Coding & Implementation:**
   - In implementation with input from system design, the system design is developed in small program is units and each units is tested for its functionality.
   - Engineers start creating the entire system by crafting code using the required technology.

5. **Integration & Unit Testing**:
   - Small programs called units, and each unit are testing after, after that all units are combined that you called as integration testing.
   - All units are integrated & test complete product for any false & failure in this phase is called testing.

6. **Deployment & Maintenance**:
   - Deployment is nothing but ones functional & non-functional testing is done then ready delivery the product to the customer or into the market.
   - Once the system is deployed, any necessary upgrades, enhancements, and changes can be made, implementing new features into the operating software.

Different types of SDLC Models are:



Software engineering ethics:

Software engineering ethics refers to the principles and guidelines that govern the behavior of software engineers in their professional practices.

1. **Quality and Safety** : Software engineers have a responsibility to produce high-quality, reliable, and secure software. They should prioritize the safety and well-being of users and the public. This includes thorough testing, validation, and verification

2. **Honesty and Integrity** : Software engineers should be honest and transparent in their work. This involves providing accurate information about the capabilities and limitations of software. Being honest about project timelines, and communicating potential risks to stakeholders. Integrity is crucial in maintaining trust with clients, users, and the public.

3. **Professional Competence**: Software engineers should strive to maintain and enhance their professional skills and knowledge throughout their careers. Staying current with advancements in technology.

4. **Respect for Privacy** : Software engineers often work with sensitive data. Respecting the privacy of individuals and organizations is critical.

5. **Transparency and Accountability**: Engineers should be transparent about the functionality and purpose of their software. Users should be informed about how their data is used and have the ability to understand and control the software's behavior.
If errors or flaws are discovered, software engineers should take responsibility for fixing them promptly.

6. **Environmental Impact**: Consideration should be given to the environmental impact of software development and usage. This includes energy consumption, electronic waste, and sustainable practices.

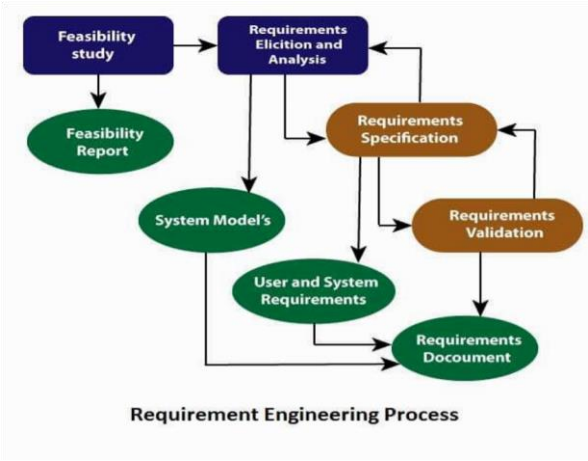
7. **Legal Compliance**: Engineers should be aware of and comply with relevant laws and regulations related to software development, including but not limited to privacy laws, accessibility standards, and industry-specific regulations.

8. **Open Communication**: Effective communication is crucial in software engineering. Engineers should communicate openly and honestly with colleagues, clients, and other stakeholders.

Requirements engineering:

Requirements engineering is a crucial phase in the software development lifecycle that involves the systematic elicitation, analysis, specification, validation, and management of requirements.

Requirement Engineering Process:



Requirement Engineering Process

- Feasibility Study
- Requirement Elicitation and Analysis
- Software Requirement Specification
- Software Requirement Validation
- Software Requirement Management

1.Feasibility Study:

- The objective behind the feasibility study is to create the reasons for developing the software that is acceptable to users, flexible to change and conformable to established standards.

Types of Feasibility:

- **Technical Feasibility** - Technical feasibility evaluates the current technologies, which are needed to accomplish customer requirements within the time and budget.
- **Operational Feasibility** - Operational feasibility assesses the range in which the required software performs a series of levels to solve business problems and customer requirements.
- **Economic Feasibility** - Economic feasibility decides whether the necessary software can generate financial profits for an organization.

2. Requirement Elicitation and Analysis:

- This is also known as the gathering of requirements. Here, requirements are identified with the help of customers and existing systems processes, if available.

3. Software Requirement Specification:

- Software requirement specification is a kind of document which is created by a software analyst after the requirements collected from the various sources - the requirement received by the customer written in ordinary language. It is the job of the analyst to write the requirement in technical language so that they can be understood and beneficial by the development team.
- The models used at this stage include ER diagrams, data flow diagrams (DFDs) and etc.

4. Software Requirement Validation:

- After requirement specifications developed, the requirements discussed in this document are validated.
- The user might demand illegal, impossible solution or experts may misinterpret the needs.

5. Software Requirement Management:

- Requirement management is the process of managing changing requirements during the requirements engineering process and system development.
- New requirements emerge during the process as business needs a change, and a better understanding of the system is developed.

6. Software Requirement Validation:

- After requirement specifications developed, the requirements discussed in this document are validated.
- The user might demand illegal, impossible solution or experts may misinterpret the needs.

7. Software Requirement Management:

- Requirement management is the process of managing changing requirements during the requirements engineering process and system development.
- New requirements emerge during the process as business needs a change, and a better understanding of the system is developed.

Benefits of Requirements Engineering:

1. Risk Reduction
2. Cost and Time Savings
3. Improved Communication
4. Customer Satisfaction
5. Quality Assurance

Functional & non-functional requirement.

Functional Requirements: Functional requirements define the specific functionalities or features that a software system must possess. These requirements describe what the system should do in terms of input, processes, and output.

- Functional requirements form the behavior of the system. If someone is giving you functional requirements for a project, they're giving you information on how the project's product is going to work.
- These requirements describe the interactions between a system and its environment.
- The functional requirements for a system describe what the system should do.
- These requirements depend on the type of software being developed, the expected users of the software, and the general approach taken by the organization when writing requirements.
- When expressed as user requirements, functional requirements are usually described in an abstract way that can be understood by system users. Functional system requirements vary from general requirements covering what the system should do to very specific requirements reflecting local ways of working or an organization's existing systems.
- Examples for functional requirements for MHC-PMS system includes:
- A user shall be able to search the appointments lists for all clinics.
- The system shall generate each day, for each clinic, a list of patients who are expected to attend appointments that day.
- Each staff member using the system shall be uniquely identified by his or her eight- digit employee number. The functional requirements specification of a system should be both complete and consistent.
- Completeness means that all services required by the user should be defined.
- Consistency means that requirements should not have contradictory definitions

Non-Functional Requirements: Non-functional requirements specify the qualities or characteristics that describe how well the system performs its functions. Unlike functional requirements, nonfunctional requirements are not concerned with specific behaviors but rather with overall system attributes.

1. **Performance**: Non-functional requirements include aspects related to the system's speed, responsiveness, and throughput.

2. **Reliability**: Address the system's ability to consistently perform its functions without failures or errors.
3. **Usability**: Focus on the user-friendliness and overall user experience of the system.

4. **Scalability**: Describes the system's ability to handle increased load, data volume, or user base without significant performance degradation 5. Security: Specifies the measures and features in place to protect the system from unauthorized access, data breaches, or other security threats.

5. **Availability**: Refers to the proportion of time the system is operational and accessible to users.

6. **Maintainability**: Describes how easily the system can be modified, updated, or repaired without causing disruptions.

classification of non-functional requirements.

1. **Product Requirements:**

• These requirements specify or constrain the behavior of the software.

• Examples include performance requirements on how fast the system must execute and how much memory it requires, reliability requirements that set out the acceptable failure rate, security requirements, and usability requirements.

2. **Organizational Requirements:**

• * These requirements are broad system requirements derived from policies and procedures in the customer's and developer's organization.

• * Examples include operational process requirements that define how the system will be used, development process requirements that specify the programming language, the development environment or process standards to be used, and environmental requirements that specify the operating environment of the system.

3. **External requirements:**

• * This broad heading covers all requirements that are derived from factors external to the system and its development process.

• * These may include regulatory requirements that set out what must be done for the system to be approved for use by a regulator, such as a central bank.

The Software Requirements Document

The software requirements document (sometimes called the software requirements specification or SRS) is an official statement of what the system developers should. It should include both the user requirements for a system and a detailed specification of the system requirements .The requirements document has a diverse set of users, ranging from the senior management of the organization that is paying for the system to the engineers responsible for developing the software

Software Requirement Specification (SRS): - **IEEE structure of a requirements document (SRS)**

The Software Requirements Specification (SRS) is a key document in software engineering that outlines the requirements for a software system. The IEEE (Institute of Electrical and Electronics Engineers) has established a standard structure for the SRS to ensure consistency and completeness.

The IEEE structure of the Software Requirements Document provides a systematic and organized way to capture and communicate the requirements for a software system, ensuring that all stakeholders have a clear and understanding of the project's goals and specifications.

Detailed explanation of the IEEE structure of a Software Requirements Document:

7.2 FUTURE SCOPE 8. BIBLIOGRAPHY

8.1 REFERENCES

Requirement elicitation and analysis:

Requirements elicitation is the process of gathering actual requirements about the needs and expectations of stakeholders for a software system.

To understand customers, business manuals, the existing software of same type, standards and other stakeholders of the project.

Techniques of Requirements Elicitation:

1.Interviews: One-on-one conversations with stakeholders to gather information about their needs and expectations.

2.Surveys: These are questionnaires that are distributed to stakeholders to gather information.

3.Focus Groups: Small groups of stakeholders who are brought together to discuss their needs.

4.Observation: Observing the stakeholders in their work environment to gather information.

5.Prototyping: Creating a working model of the software system, this can be used to gather feedback from stakeholders and to validate requirements.

Software Requirement Validation:

- After SRS developed, the requirements discussed in this document are validated or tested.

Requirement validation done through Requirement reviews taken by customers, after developing prototyping check with customers & Test case generations.

Requirements can be the check against the following conditions

- If they are correct and as per the functionality and especially of software.
- Any changing or additional requirement.
- If they can practically implement.

These checks include:

1. **Validity Checks**: A user may think that a system is needed to perform certain functions.

2. **Consistency Checks**: Requirements in the document should not conflict. That is, there should not be contradictory constraints or different descriptions of the same system function.

3**. Completeness Checks**: The requirements document should include requirements that define all functions and the constraints intended by the system user.

4. **Realism Checks**: Using knowledge of existing technology, the requirements should be checked to ensure that they can actually be implemented.

5. **Verifiability**: To reduce the potential for dispute between customer and contractor, system requirements should always be written so that they are verifiable. This means that you should be able to write a set of tests that can demonstrate that the delivered system meets each specified requirement.

There are a number of requirements validation techniques that can be used individually or in conjunction with one another

1. **Requirements Reviews**: The requirements are analyzed systematically by a team of reviewers who check for errors and inconsistencies.

2. Prototyping: In this approach to validation, an executable model of the system in question is demonstrated to end-users and customers. They can experiment with this model to see if it meets their real needs.

3. Test-Case Generation: Requirements should be testable. If the tests for the requirements are devised as part of the validation process, this often reveals requirements problems.

Software Requirement Management:

Requirement management is the process of managing changing requirements during the requirements engineering process.

Activities involved in Requirement Management:

1. Tracking & Controlling Changes: Handling changing requirements from customers.
2. Version Control: Keeping track of different versions of system.
3. Traceability: Trace to software is design, develop or test as per requirements or not.
4. Communication: If changing requirements has any issues contact with clients within time.
5. Monitoring & Reporting: Monitoring development process & report the status.

Requirements Management Planning

1. **Requirements Identification**:
   Each requirement must be uniquely identified so that it can be cross referenced with other requirements & used in traceability assessments
2. **A Change Management Process**:

* This is the set of activities that assess the impact and cost of changes.

3. **Traceability Policies:**

* These policies define the relationships between each requirement and between the requirements and the system design that should be recorded.

* The traceability policy should also define how these records should be maintained.

4. **Tool Support**:

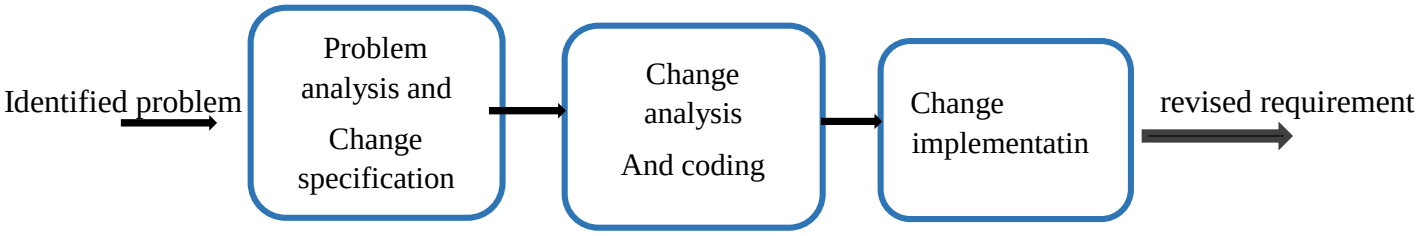
* Requirements management involves the processing of large amounts of information about the requirements.
* Tools that may be used range from specialist requirements management systems to spreadsheets and simple database systems.

*** Tool supports might be needed for**:

a. Requirements Storage: The requirements should be maintained in a secure, managed data store that is accessible to everyone involved in the requirements engineering process.

b. Change Management: The process of change management is simplified if active tool support is available as shown in fig .

c. Traceability Management: Tool support for traceability allows related requirements to be discovered. Some tools are available which use natural language processing techniques to help discover possible relationships between requirements.



There are three principal stages to a change management process:

1. Problem Analysis and Change Specification:
* The process starts with an identified requirements problem or, sometimes, with a specific change proposal.
* During this stage, the problem or the change proposal is analyzed to check that it is valid. This analysis is fed back to the change requestor who may respond with a more specific requirements change proposal, or decide to withdraw the request.

2. Change Analysis and Costing:
* The effect of the proposed change is assessed using traceability information and general knowledge of the system requirements.

3. Change Implementation:
* The requirements document and, where necessary, the system design and implementation, are modified.
* Requirements document will have to be organized so that changes can be made to it without extensive rewriting or reorganization